



Documentation Technique

CNRS

Amélioration TimeLineJs



Sommaire

Sommaire	1
Version du document	1
Mise en avant des catégories.	1
Initialisation du style.	1
Gérer l'ordre des catégories et leurs couleurs.	2
Attribution de l'ordre et couleur des catégories.	2
Récupération des valeurs.	3
Création des groupes.	3
Attribution Couleur.	5
Gérer la couleur des markers.	5
Initialisation.	5
Mise en avant des événements et ses impacts.	5
Relation entre Parent Enfant.	5
Récupération information.	6
Interaction clic.	7
Gestion opacité marker.	7
Relation distante.	8
La ToolTip.	8
On Blur.	10
Mise en place d'un zoom lors d'un clic.	10
Mise en place.	10
Récupération.	11
Mise en place d'un filtrage d'événement	12
Configuration Timeline.	12
Mise en place html.	12
Valeur de filtrage par thème.	12
Chargement uniquement des éléments filtrés.	12

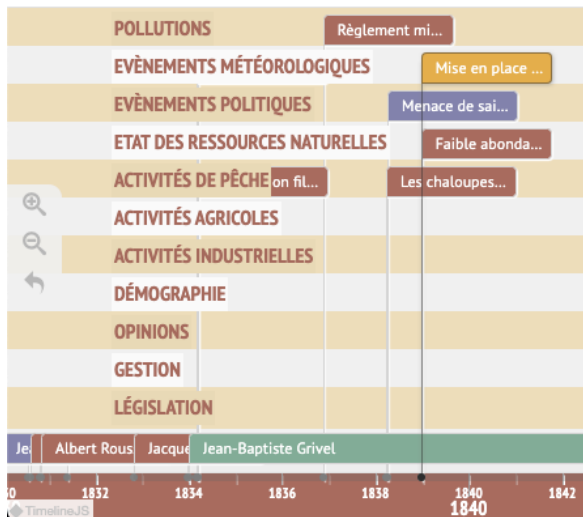
Version du document

1.0	version initiale	J.Doussain, L.Roger	09-2022
1.1	Ajout de filtrage	J.Doussain, L.Roger	09-2024

Mise en avant des catégories.

Initialisation du style.

Mise en avant des catégories en les passant au premier plan.



Résultat obtenu en ajoutant un `padding` sur chacune des catégories.

Changement de couleur de fond d'écran du texte reprenant la couleur de la ligne du fond.

Assombrissement de la couleur de fond pour être en adéquation avec les lignes du fond.

Augmentation du `z index` `TL-TimeGroup.less` ligne 27

Code modifié :

Fichier `TimeGroup.js`

```
87  _initLayout () {
88
89      // Create Layout
90      this._el.message = DOM.create("div", "tl-timegroup-message ", this._el.container);
91
92      //Handle Color categories
93      this._el.message.style.backgroundColor = this.data.color;
94      this._el.message.style.filter = "brightness(98%)";
95      this._el.message.style.padding = "2px 2px 3px 2px";
96
97      this._el.message.innerHTML = this.data.label;
98  }
```

Gérer l'ordre des catégories et leurs couleurs.

Attribution de l'ordre et couleur des catégories.

Les catégories ainsi que leur couleur et ordre sont à définir au début du Google Sheet.

Q	R	S
Group	GroupColor	GroupOrder
Législation	#EFE1C2	11
Gestion		10
Opinions	#EFE1C2	9
Etat des ressources naturelles		4
Activités de pêche	#EFE1C2	5
Activités agricoles		6
Activités industrielles	#EFE1C2	7
Démographie		8
Pollutions	#EFE1C2	1
Evènements météorologiques		2
Evènements politiques	#EFE1C2	3

Récupération des valeurs.

Fichier TimeScale.js

```
if (slides[i].group) {  
  if (groups.indexOf(slides[i].group) < 0) {  
    var colorLigne = slides[i].GroupColor;  
    var group_order = slides[i].GroupOrder;  
    colors.push(colorLigne);  
    group_orders.push(group_order);  
    groups.push(slides[i].group);  
  }  
}
```

Pour chaque groupe existant, on ajoute sa couleur et son ordre.

`Group_info` pourra être accessible dans le fichier `TimeNav` qui nous permettra de modifier la Timeline.

Une fois les valeurs définies dans le GoogleSheet, elles sont récupérées et attribuées dans 2 tableaux `colors` et `group_orders`.

```
for (var i = 0; i < groups.length; i++) {  
  group_info[i] = {  
    label: groups[i],  
    color: colors[i],  
    order: group_orders[i],  
    idx: i,  
    positions: [],  
    n_rows: 1, // default  
    n_overlaps: 0  
  };  
}
```

Création des groupes.

Fichier TimeNav.js

```
_createGroups() {  
  this._groups = [];  
  var group_labels = this.timescale.getGroupLabels();  
  
  var group_ordered = this._reorderGroup(group_labels);  
  if (group_labels) {  
    this.options.has_groups = true;  
    for (var i = 0; i < group_ordered.length; i++) {  
      this._createGroup(group_ordered[i]);  
    }  
  }  
}
```

Nous récupérons les valeurs des groupes via le fichier TimeScale vu avant.

Group_labels détient maintenant nos groupes avec leur ordre et couleur.

Les groupes ne sont pas encore ordonnés.

Création de group_ordered en appelant _reorderGroup.

La fonction _reorderGroup prend en paramètre nos groupes et les réordonne.

Une fois réorganisés, nos groupes sont créés dans le fichier TimeGroup.js

```
_reorderGroup(groups) {  
  var group_ordered = [];  
  //sort groups by group_order  
  for (var i = 0; i < groups.length; i++) {  
    var group = groups[i];  
    if (group.group_order) {  
      group_ordered[group.group_order - 1] = group;  
    }  
  }  
  return group_ordered;  
}
```

Fichier TimeMarker.js

```
_initLayout() {  
  
  // Create Layout  
  let isHidden = this.data.GroupOrder ? 'hidden' : '';  
  this._el.container = DOM.create("div", `tl-timemarker ${isHidden}`);
```

Les groupes étant considérés comme des markers doivent être cachés.

Donc si marker contient GroupOrder, on ne l'affiche pas.

Attribution Couleur.

Fichier TimeGroup.js

```
// Data  
this.data = {  
  label: "",  
  rows: 1,  
  color: ""  
};
```

On ajoute l'attribut couleur à la classe TimeGroup.

Puis, on y accède via this.data.color.

```
// Change the group line color  
this._el.container.style.backgroundColor = this.data.color;
```

Gérer la couleur des markers.

Initialisation.

La couleur du marker est définie en hexadécimal dans le GoogleSheet, colonne MarkerColor.

MarkerColor
#8db5a1
#b18869
#b16972
#b16972
#b18869
#b18869
#8db5a1
#8db5a1
#8db5a1
#8db5a1

Fichier ConfigFactory.js

```
function extractEventFromCSVObject(orig_row) {  
  let row = {}  
  Object.keys(orig_row).forEach(k => {  
    row[k] = trim(orig_row[k]) // get rid of spaces  
  })  
  var d = {  
    id: row['n°ID'] || '',  
    type: row['Type'] || '',  
    categories: row['Categorie'] || '',  
    markerColor: row['MarkerColor'] || '',  
    GroupOrder: row['GroupOrder'] || '',  
    GroupColor: row['GroupColor'] || '',  
    parentOf: row['parentOf'] || '',  
    childOf: row['childOf'] || '',  
    typeOfLink: row['TypeOfLink'] || '',  
    ZoomOnClick: row['ZoomOnClick'] || ''  
  }  
}
```

Une fois définie, on ajoute une propriété dans l'objet marker.

Cette propriété prend la valeur de `row['MarkerColor']` cela correspond au champ MarkerColor dans GoogleSheet.

Si la valeur n'existe pas, la case sera grise par défaut.

Fichier TimeMarker.js

Lors de l'initialisation du layout et donc des markers, on change la couleur du fond.

```
// Handle color  
this.data.markerColor !== "" ? this._el.content_container.style.backgroundColor = this.data.markerColor : "";
```

Mise en avant des événements et ses impacts.

Relation entre Parent Enfant.

Pour pouvoir mettre en avant les événements et leur impact, on crée une relation entre plusieurs éléments avec les colonnes parentOf, ChildOf en listant les identifiants

parentOf	childOf
	E0035
E0034,E0036	
	E0035

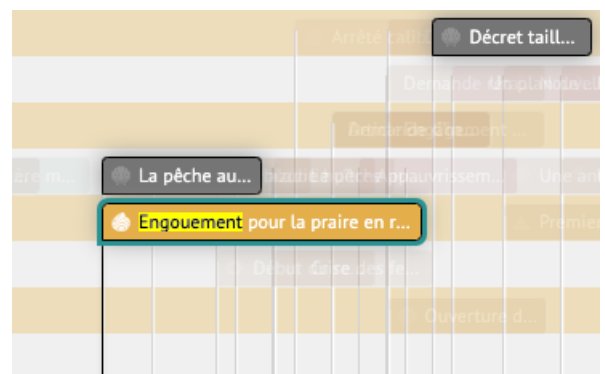
Un parent peut avoir plusieurs enfants.

Il faut séparer les identifiants par une **virgule** ou un **point-virgule**.

Lors du clic sur un marker parent, tous les autres marker seront cachés sauf ses enfants.

Vice versa lors du clic sur un marker enfant.

Il affichera seulement son parent ainsi que les enfants du parent.



Récupération information.

Fichier ConfigFactory.js

```
85     parentOf: row['parentOf'] || '',
86     childOf: row['childOf'] || '',
87
88     if(d.parentOf != ''){
89         var parents = d.parentOf.split(/[,;]/);
90         d.parentOf = parents;
91     }
92     if(d.childOf != ''){
93         var children = d.childOf.split(/[,;]/);
94         d.childOf = children;
95     }
```

Les champs sont récupérés dans le GoogleSheet puis affectés à **parentOf** et **childOf**.

Une fois récupérés, ils sont traités pour séparer les valeurs, via une virgule ou un point-virgule.

Interaction clic.

Fichier TimeMarker.js

```
_onMarkerClick(e) {  
    //Remove highlight from other markers  
    $(".tl-timemarker-content-container").removeClass('highlighted');  
    $(this._el.content_container).addClass('highlighted');
```

On ajoute une class **highlighted** au marker cliqué pour le faire ressortir.

Ensuite, nous venons mettre en surbrillance (highlight) tous les markers contenus dans les **parentOf** et **childOf**

```
//highlight the parent and its children  
this._setHighlight(this.data.parentOf);  
this._setHighlight(this.data.childOf);
```

La fonction prend en paramètre **data**, boucle sur **data**.

Pour chaque parent, enfant, il enlève les espaces et ajoute **highlighted**.

```
_setHighlight(data) {  
    for (var i = 0; i < data.length; i++) {  
        //remove space  
        var str = data[i].replace(/\s/g, '');  
        $(".." + str).addClass("highlighted");  
    }  
}
```

“.”+**str** Fait référence au marker, en effet lors de la création du marker, on affecte l'id contenu dans le GoogleSheet à une id Marker

```
let idMarker = this.data.id;  
idMarker = idMarker.replace(/\s/g, '');  
this._el.content_container = DOM.create("div", `tl-timemarker-content-container ${idMarker} ${hasParent} ${isHidden}`, this._el.container);
```

Gestion opacité marker.

Lors du clic, l'opacité change seulement si le marker contient des enfants ou parents.

```
//Change opacity if marker has childs or parents  
if (!this.data.parentOf.includes('') || !this.data.childOf.includes('')) {  
    $(".tl-timemarker-content-container").not('.highlighted').not('.tl-timemarker-active').css('opacity', 0.1);  
}
```

Relation distante.

Si un utilisateur clique sur un marker enfant, il faut afficher les markers enfants de son parent.

Cela est réalisé grâce à cette méthode `markerclick` qui nous renvoie sur la méthode de `TimeNav.js`

Fichier `TimeMarker.js`

```
//Fire event that this marker has been selected
this.fire("markerclick", { unique_id: this.data.unique_id, zoomLevel: this.data.ZoomOnClick ,parent: this.data.childOf });
```

Une fois dans `TimeNav.js` `_onMarkerClick` est appelé.

Fichier `Timenav.js`

```
marker.on('markerclick', this._onMarkerClick, this);
```

```
_onMarkerClick(e) {
  if (e.parent) {
    let parents = this._removeBlankSpace(e.parent);

    // Find the child of the parent and highlight it
    for (var i = 0; i < parents.length; i++) {
      let index = this._markers.findIndex(x => x.data.id == parents[i]);
      if(index == -1){
        //skip loop
        continue;
      }
      let childs = this._markers[index].data.parentOf;
      if (childs.length > 0) {
        childs.map((child) => {
          //clear space
          var str = child.replace(/\s/g, '');
          $('.' + str).addClass('highlighted');
        })
      }
    }
  }
}
```

Cette méthode cherche pour chaque parent, les enfants de celui-ci.

Ensuite, il applique pour chaque enfant une classe `highlighted`.

La Tooltip.

Si un marker est cliqué, une tooltip apparaîtra si l'on survol (hover) ses enfants.
Le texte de la tooltip est à définir dans le GoogleSheet dans la colonne TypeOfLink.

n°ID	parentOf	childOf	TypeOfLink
E0034		E0035	Conséquence
E0035	E0034,E0043		Cause indirecte
E0036		E0035	Cause direct



Dans cet exemple :
Engouement pour.... est E0035
La pêche aux coquilles... est E0034
Décret taill... est E0036

Fichier TimeMarker.js

Pour pouvoir créer une tooltip, on utilisera la fonction `_initiateToolTip` avec les parents et enfants concernés.

```
//initialization of tooltip on only parent and child  
this._initiateToolTip(this.data.parentOf);  
this._initiateToolTip(this.data.childOf);
```

```
_initiateToolTip(data) {  
  if (!Array.isArray(data)) {  
    return;  
  }  
  data.forEach(element => {  
    var str = element.replace(/\\s/g, '');  
    $("." + str).mouseenter(function () {  
      $("." + str).removeClass('tl-tooltip-hidden').addClass('tl-tooltip');  
    });  
    $("." + str).mouseleave(function () {  
      setTimeout(function () {  
        $("." + str).removeClass('tl-tooltip').addClass('tl-tooltip-hidden');  
      }, 1000);  
    });  
  });  
}
```

La fonction change la classe de la tooltip des enfants, parents si la souris passe dessus.

De même si la souris en sort.

Le changement se fait via les classes `tl-tooltip` et `tl-tooltip-hidden`

Mise en place de la tooltip sur chaque élément possédant un TypeOfLink

```
if (this.data.typeOfLink !== "") {
    this._el.tooltip = DOM.create("div", `tl-tooltip-hidden tooltip-${this.data.id}`, this._el.timespan);
    this._el.tooltip.innerHTML = this.data.typeOfLink;
}
```

On Blur.

Lorsqu'un utilisateur clique sur un autre marker ou bien en dehors, on vient réinitialiser les markers, les tooltips, et l'opacité.

Ces actions sont réalisées grâce à la fonction `_onMarkerBlur` voire ci-dessous.

Fichier `TimeMarker.js`

```
_onMarkerBlur(e) {
    this.fire("markerblur", { unique_id: this.data.unique_id });
    $(".tl-timemarkers-content-container").css('opacity', 1);
    $(".tl-timemarkers-content-container").removeClass('highlighted');
    this._deinitiateTooltip(this.data.parentOf);
    this._deinitiateTooltip(this.data.childOf);
}
```

```
_deinitiateTooltip(data) {
    if (!Array.isArray(data)) {
        return;
    }
    data.forEach(element => {
        data.forEach(element => {
            var str = element.replace(/s/g, '');
            $("." + str).unbind('mouseenter');
        });
    });
}
```

La fonction `_deinitiateTooltip` permet d'enlever les binds de `mouseenter` et `mouseleave` des tooltips devenu inutile.

Sans cela les tooltips resteraient affichés en survol même après avoir choisi un autre marker.

Mise en place d'un zoom lors d'un clic.

Mise en place.

Pour pouvoir zoomer lors d'un clic, on utilise une nouvelle colonne ZoomOnClick qui définit le niveau de zoom associé à l'item.

ZoomOnClick
10
2
4
10

Par défaut, l'intervalle de valeur de zoom est entre 0 et 10

10 représentant le niveau de zoom le plus élevé.

Récupération.

Fichier ConfigFactory.js

Comme pour le reste, nous venons définir ZoomOnClick pour la classe TimeMarker

```
ZoomOnClick: row['ZoomOnClick'] || '',
```

Fichier TimeMarker.js

Comme pour la relation distante, on utilise la méthode `fire` avec `this.data.ZoomOnClick` en paramètre.

```
//Fire event that this marker has been selected  
this.fire("markerClick", { unique_id: this.data.unique_id, zoomLevel: this.data.ZoomOnClick ,parent: this.data.childOf });
```

Puis l'on rajoute dans `_onMarkerClick` le niveau de zoom voulu.

```
// Set the zoom level  
if (e.zoomLevel != "") {  
    this.setZoom(e.zoomLevel);  
}
```

Mise en place d'un filtrage d'événement

Configuration Timeline.

Fichier `Timeline.js`

Dans le constructeur de Timeline ajout de :

“enable_filter: false,”

Mise en place html.

Ajout au container d'une div “tl-filterbar”

```
590         if (this.options.enable_filter) {
591             this._el.filterbar = DOM.create(
592                 "div",
593                 "tl-filterbar",
594                 this._el.container
595             );
596         }
597     }
```

Il faut maintenant créer le formulaire :

```
this._el.filterbar.innerHTML = `
<div class="container">
  <div class="field-group" style="margin: 2px">
    ....
  </div>
</div>`
```

Valeur de filtrage par thème.

Mise en place du select theme :

Il faut pouvoir récupérer les valeurs des thèmes pour pouvoir les définir dans les options du select.

```

if (this.options.enable_filter) {
  let select = this._el.filterbar.querySelector("#FilterTheme");
  let themes = this._timenav._groups.map((group) => group.data.label);
  themes.forEach((theme) => {
    let option = document.createElement("option");
    option.text = theme;
    option.value = theme;
    if (inputFilter.theme === theme) {
      option.selected = true;
    }
    select.add(option);
  });
}

```

Chargement uniquement des éléments filtrés.

Si des filtres sont remplis, on ne va filtrer que sur ces filtres.

```

_initFilteredData(data, filter) {
  if (typeof data === "string") {
    makeConfig(data, {
      callback: function (config) {
        this.setConfigFilter(config, filter);
      }.bind(this),
      sheets_proxy: this.options.sheets_proxy,
    });
  } else if (data instanceof TimelineConfig) {
    this.setConfigFilter(data, filter);
  } else {
    this.setConfigFilter(new TimelineConfig(data), filter);
  }
}

```